

Maze Escape: Human vs Monster AI Survival Game

Group Number: XX

Member 1	Member 2	Member 3
Member 4	Member 5	Member 6

June 2026

Course Code: SOF106

Course Name: Principles of Artificial Intelligence

Academic Session: 202604

Assessment Title: Final Assessment Group Project

Contents

1	Introduction	3
2	Project Objectives	3
3	Methodology	3
3.1	Map Representation	3
3.2	Genetic Algorithm for Map Generation	4
3.3	Human Movement and BFS	4
3.4	Monster AI Algorithms	4
3.5	Item System	4
4	Implementation	5
5	Validation and Verification	5
5.1	Map Generation Test	5
5.2	Spawn Test	5
5.3	Boundary Test	5
5.4	Algorithm Comparison	5
6	Results and Discussion	5
7	Conclusion	6

Abstract

This project develops an AI-based survival chase game called Maze Escape. The game is built on a 30 by 30 grid map where the human moves one step per turn and the monster moves two steps per turn. The project applies several artificial intelligence algorithms, including Genetic Algorithm, BFS, Greedy Search, A* Pathfinding, Minimax, and Simulated Annealing. The aim of this project is to demonstrate how AI algorithms can be used in a dynamic game environment.

1 Introduction

Maze Escape is a human versus monster survival game. The player needs to survive in a maze environment while monsters try to catch the human. This project was selected because pathfinding and decision-making are important topics in artificial intelligence, especially in game AI.

The game uses a 30 by 30 grid-based map. The map supports wrap-around boundary movement, which means that when a character moves out of one side of the map, it appears on the opposite side. This rule makes the pathfinding problem more interesting because AI algorithms need to consider the map as a continuous grid.

2 Project Objectives

The main objectives of this project are:

- To develop a playable human versus monster survival game.
- To apply artificial intelligence algorithms in a game environment.
- To generate playable maps using a Genetic Algorithm.
- To implement different monster AI strategies.
- To compare the behavior and performance of different algorithms.

3 Methodology

3.1 Map Representation

The game map is represented as a 30 by 30 two-dimensional grid. In the map, 0 represents a walkable floor cell, while 1 represents a wall. The human and monsters can only move on walkable cells.

The map also uses a wrap-around boundary system. For example, if a character moves left from column 0, it will appear at column 29. This is implemented by using the modulo operation.

3.2 Genetic Algorithm for Map Generation

A Genetic Algorithm is used to generate playable maps. Each map is treated as one chromosome. A chromosome is represented as a 30 by 30 binary grid, where 0 means floor and 1 means wall.

The initial population contains several randomly generated maps. Each map is evaluated using a fitness function. The fitness function considers reachability, wall ratio, junction complexity, and path distance. Reachability checks whether most walkable cells are connected. Wall ratio prevents the map from being too empty or too blocked. Junction complexity encourages more branching paths. Path distance helps keep the chase distance reasonable.

After fitness evaluation, tournament selection is used to choose better maps as parents. Crossover combines two parent maps by cutting and joining their rows or columns. Mutation randomly changes a small number of cells from floor to wall or from wall to floor. After several generations, the best map is saved as a text file using 0 and 1 values separated by spaces.

3.3 Human Movement and BFS

The human moves one step per turn. BFS can be used to search reachable cells and support escape path planning. Since the map is an unweighted grid, BFS is suitable for finding shortest paths.

3.4 Monster AI Algorithms

Several AI algorithms are used for monster movement.

Greedy Search allows the monster to choose the next move that reduces the distance to the human. It is simple and fast, but it may not perform well when obstacles exist.

A* Pathfinding uses the formula:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual movement cost from the start node, and $h(n)$ is the estimated distance to the target. A* is used to find an efficient path from the monster to the human.

Minimax treats the chase as a two-player decision problem. The monster tries to minimize the distance to the human, while the human tries to maximize the distance from the monster.

Simulated Annealing is used as a stochastic search strategy. It explores possible movement choices and may accept worse solutions with a certain probability to avoid local optima.

3.5 Item System

Items are generated after a fixed number of human steps. For example, new items may appear every 10 human steps. Unused items remain on the map. Items can affect the movement and strategy of the game, such as giving the human a speed boost or temporarily stopping the monster.

4 Implementation

The project is divided into several modules. The AI folder stores the algorithm implementations, including Genetic Algorithm, BFS, Greedy Search, A*, Minimax, and Simulated Annealing. The game folder stores the main game logic, including map handling, player movement, monster movement, item logic, collision checking, and scoring.

The Genetic Algorithm module outputs a text file containing the generated map. The game system can then load this map and use it during gameplay.

5 Validation and Verification

Several tests are used to validate the project.

5.1 Map Generation Test

The generated map should have 30 rows and 30 columns. Each cell should only contain 0 or 1. The wall ratio should be reasonable, and most walkable cells should be connected.

5.2 Spawn Test

The human and monster should spawn on walkable cells. They should not spawn on the same position. Their starting distance should also be within a reasonable range to avoid unfair gameplay.

5.3 Boundary Test

The wrap-around boundary is tested by moving characters across the edges of the map. Moving left from column 0 should move the character to column 29. Moving up from row 0 should move the character to row 29.

5.4 Algorithm Comparison

The monster AI algorithms can be compared using capture steps, runtime, and behavior. Greedy Search is simple and fast. A* is more stable for pathfinding. Minimax can predict future movement but requires more computation. Simulated Annealing provides more random behavior.

6 Results and Discussion

The project successfully generates playable maps and supports AI-based monster chasing behavior. The Genetic Algorithm improves map quality compared with purely random map generation. BFS can support human escape planning. A* provides more reliable chasing behavior than Greedy Search when obstacles exist.

The project also shows that different algorithms have different strengths and weaknesses. Simple algorithms are easier to run but may perform poorly in complex maps. More advanced algorithms can make better decisions but may require more computation.

7 Conclusion

In conclusion, this project demonstrates how artificial intelligence algorithms can be applied in an interactive survival chase game. The Genetic Algorithm is used for map generation, BFS is used for human path searching, and several monster AI algorithms are used for chasing behavior. Through this project, the team gained practical experience in AI search algorithms, optimization, game logic, and system integration.

Future improvements may include better multi-agent monster cooperation, more advanced item effects, improved user interface design, and reinforcement learning-based monster behavior.